AI BENCHMARKING

DASHBOARD

A local, privacy-first platform for evaluating AI models — embedding quality, LLM speed, and retrieval accuracy — entirely on your own hardware.

Python 3.8+ | React 18 | FastAPI | Open Source | MIT License

AUTHOR	REPOSITORY	DATE
naku2001	github.com/naku2001/Metrics	March 2026

01 // WHAT IS THIS TOOL?

Plain-English Overview

The AI Benchmarking Dashboard is a free, open-source web application you run on your own computer. It helps you answer one simple question: **which AI model works best for my documents?**

Instead of guessing which AI to use, or paying thousands of dollars for cloud-based evaluation tools, you upload your own PDF, let the tool generate test questions from it automatically, and then watch as multiple AI models race to answer those questions. The winner — measured by real accuracy scores — tells you exactly which model to trust with your content.

Everything runs locally on your machine. Your documents never leave your computer. There are no subscriptions, no data privacy concerns, and no black boxes.

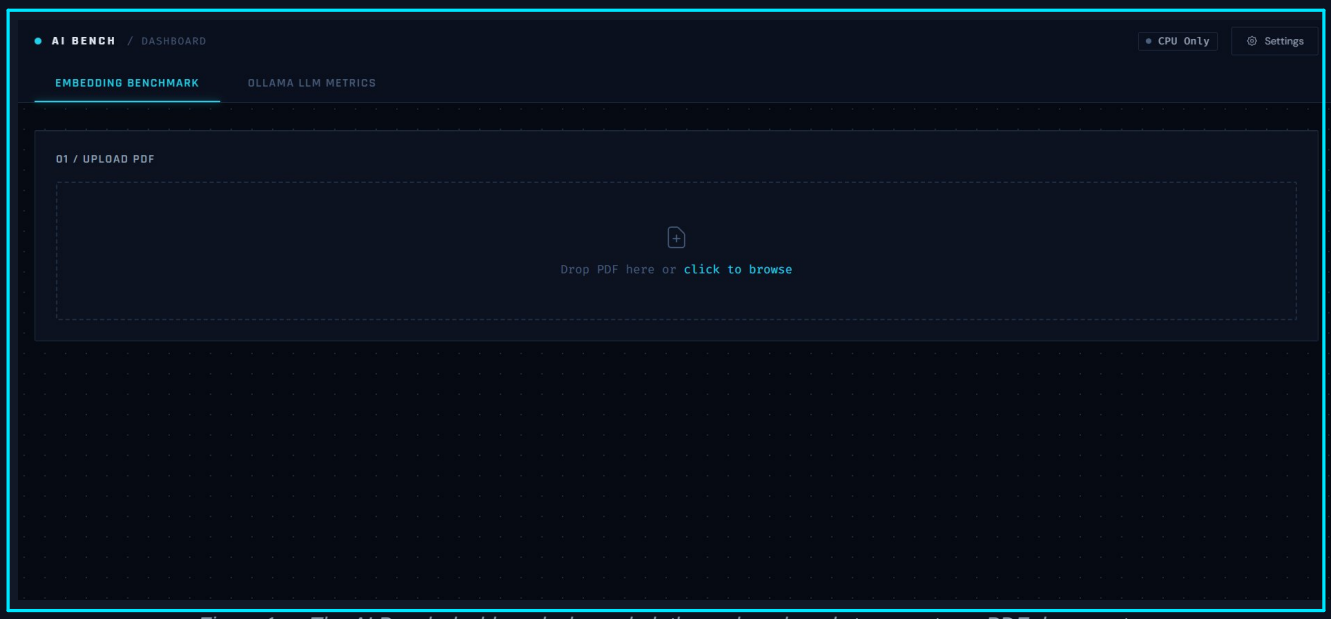

Figure 1 — The AI Bench dashboard: clean, dark-themed, and ready to accept any PDF document.

The tool has two main areas:

- **Embedding Benchmark** — Tests how well different AI models understand and retrieve text from your documents.
- **Ollama LLM Metrics** — Measures how fast locally-installed language models respond and generate text.

02 // WHO IS IT FOR?

Target Users & Use Cases

AI / ML Engineers	Need to pick the right embedding model for a RAG pipeline before committing to production infrastructure.
Data Scientists	Want objective, reproducible metrics when comparing model families (MiniLM vs BGE vs E5) on domain-specific data.
Researchers	Evaluating retrieval quality on academic papers, technical reports, or custom corpora without cloud dependencies.
Product Teams	Deciding which LLM to deploy locally for a privacy-sensitive internal tool — need speed and accuracy data together.
Indie Developers	Building a document chatbot and need to know if the small 80 MB model is good enough, or if they need the big one.
IT / Infrastructure	Assessing how well models perform on available hardware (CPU-only vs GPU) before procurement decisions.

03 // HOW THE SOLUTION WORKS

Step-by-Step Walkthrough

The Embedding Benchmark — Finding the Best Model for Your Docs

The diagram below shows the pipeline progress panel. Every run follows five automatic steps:

1 EXTRACT	The tool reads your PDF and pulls out all the text — tables, paragraphs, everything.
2 CHUNK	Long documents are split into smaller pieces (~300 words each) with a slight overlap so context is never cut off mid-sentence.
3 GENERATE QA	An AI automatically writes 3 test questions per chunk. These become the benchmark's ground truth — the 'right answers' to measure against.
4 EMBED	Each selected AI model turns every chunk into a list of numbers (a vector). This is how models 'understand' text.
5 SCORE	When a test question is asked, the tool checks which chunk each model retrieves first. Accuracy is scored using industry metrics: MRR, NDCG, and Recall.

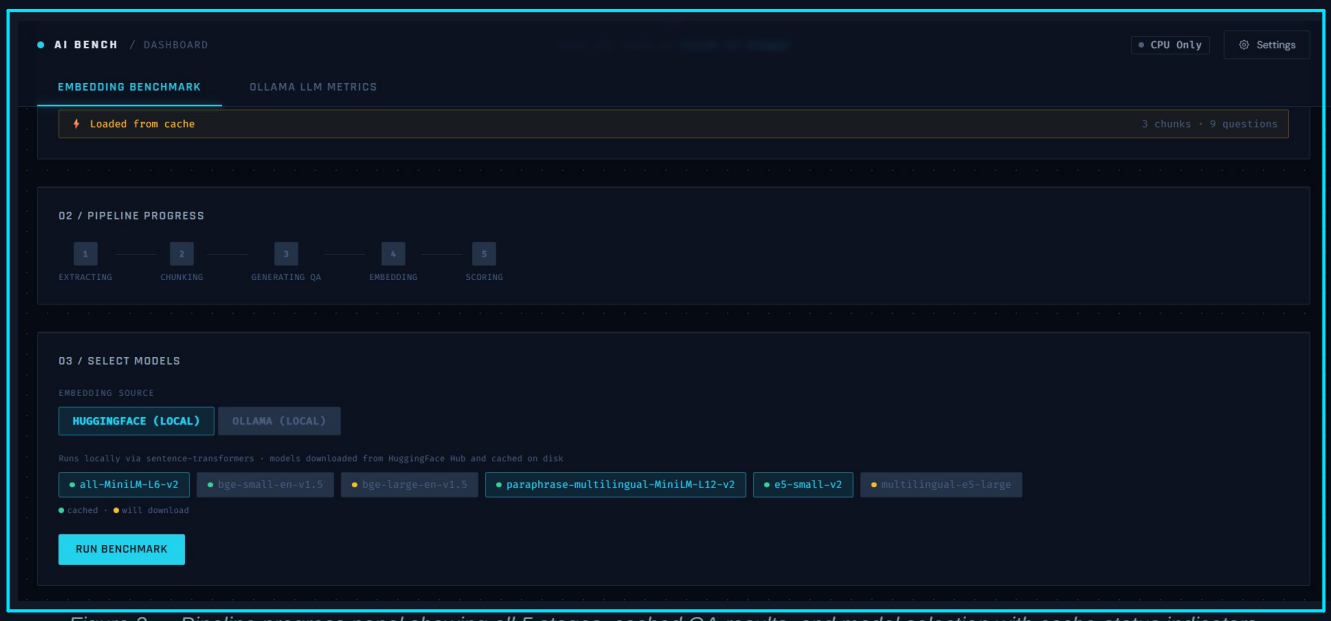

Figure 2 — Pipeline progress panel showing all 5 stages, cached QA results, and model selection with cache-status indicators.

04 // READING THE RESULTS

What the Numbers Actually Mean

After a benchmark run, the Metrics Comparison table shows side-by-side scores for every model you tested. Here is what each number means in plain English:

MRR (Mean Reciprocal Rank)	On average, was the right chunk the very first result? A score of 1.0 means it was always first. The dashboard showed all-MiniLM-L6-v2 scoring 0.94 — nearly perfect.
Recall@K	Out of all the right answers, how many did the model find within its top K results? Recall@3 of 1.0 means every correct chunk appeared in the top 3 — nothing was missed.
NDCG@K (Ranking Quality)	Did the model not just find the right chunk, but rank it higher than wrong ones? This rewards models that consistently put the best result first, not just somewhere in the list.

METRIC	all-MiniLM-L6-v2	paraphrase-multilingual-MiniLM-L12-v2	e5-small-v2
OVERALL			
MRR	0.9444	0.8889	0.8889
NDCG			
NDCG@1	0.8889	0.7778	0.7778
NDCG@3	0.9590	0.9180	0.9180
NDCG@5	0.9590	0.9180	0.9180
NDCG@10	0.9590	0.9180	0.9180
RECALL			
Recall@1	0.8889	0.7778	0.7778
Recall@3	1.0000	1.0000	1.0000
Recall@5	1.0000	1.0000	1.0000
Recall@10	1.0000	1.0000	1.0000

Figure 3 — Metrics Comparison table: all-MiniLM-L6-v2 leads with MRR 0.94 and perfect Recall@3, @5, and @10.

05 // PER-CHUNK RETRIEVAL BREAKDOWN

Seeing Exactly Where Models Succeed or Fail

Beyond the summary scores, the dashboard lets you drill into every single chunk of your document and see how each model performed on each individual question. This is the most powerful debugging feature — it tells you not just *that* a model failed, but *where* and *why*.

In the screenshot below, you can see real questions generated from a CV/resume document. Green 'rank 1' means the model immediately found the right passage. Red 'rank 2' means it was one position off — still close, but not perfect. This granular view lets you decide whether near-misses matter for your specific use case.

Figure 4 — Per-chunk breakdown showing questions auto-generated from a CV, and retrieval ranks per model. Green = rank 1 (correct), Red = rank 2 (miss).

Key insight from this view:

- e5-small-v2 ranked the 'tools and platforms' question at rank 2, while MiniLM got it right at rank 1 — showing MiniLM's advantage on technical terminology.
- All models scored identically on the programming languages chunk — a sign that chunk is easy and well-isolated.
- This view is essential for building confidence before deploying a model in production.

06 // LLM SPEED BENCHMARKING

Measuring Local Language Model Performance

The second major feature of the dashboard is the Ollama LLM Metrics tab. While the Embedding Benchmark tells you which model understands your documents best, the LLM tab tells you how fast your local language models actually generate responses.

What You Can Measure

TPS — Tokens Per Second	How many words the model generates per second. Higher is faster. A GPU might hit 80+ TPS; a CPU might do 5–15.
TTFT — Time to First Token	How long before the model starts responding. Even a fast model feels slow if TTFT is high.
Prefill TPS	How quickly the model processes your input prompt before it starts answering.
Load Time	How long it takes to load the model into memory on first use.

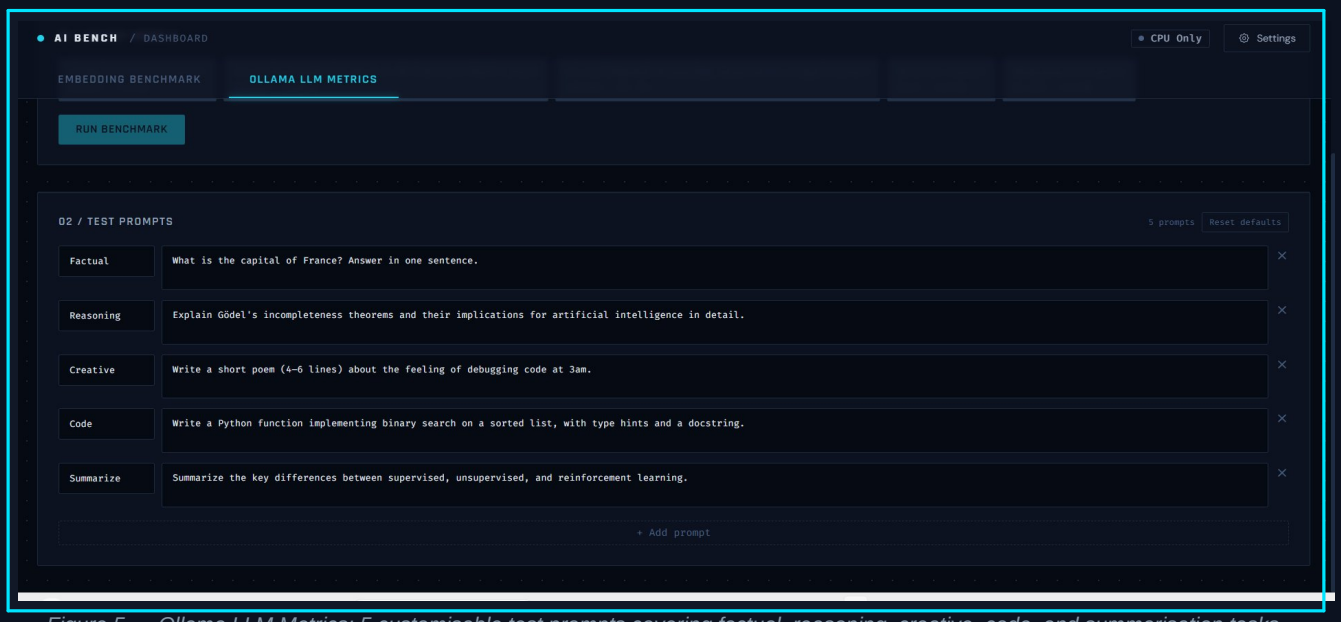

Figure 5 — Ollama LLM Metrics: 5 customisable test prompts covering factual, reasoning, creative, code, and summarisation tasks.

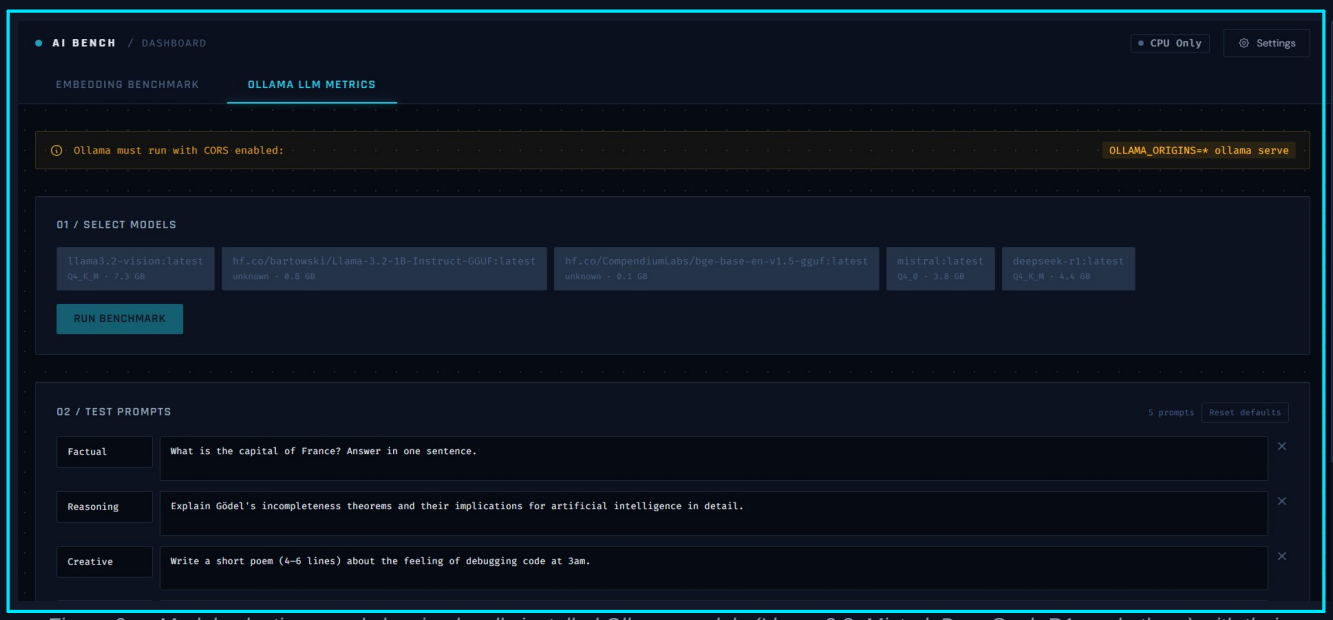

Figure 6 — Model selection panel showing locally installed Ollama models (Llama 3.2, Mistral, DeepSeek-R1, and others) with their quantisation and file sizes.

07 // BUSINESS VALUE

Why This Tool Matters

Choosing the wrong AI model for a production system is expensive. A model that looks good on generic benchmarks may perform poorly on your specific documents. This dashboard eliminates that risk by letting you test with your actual data before you build anything.

Cost Savings

Avoid paying for cloud-based evaluation APIs. All inference runs locally — the only cost is your electricity. A single cloud evaluation run can cost hundreds of dollars; this tool is free.

Data Privacy

Your documents never leave your machine. For regulated industries — healthcare, legal, finance — this is not optional, it is a requirement. No API calls, no logs, no third-party exposure.

Faster Decision Making

Instead of weeks of engineering work to set up a proper evaluation harness, the tool gives you benchmark results in minutes. Teams can make model decisions in a single meeting.

Reproducibility

Every run uses the same QA pairs (cached by PDF hash), the same scoring library (ranx), and the same prompts. Results are consistent and comparable across time and team members.

Hardware Awareness

The tool auto-detects your GPU (NVIDIA, AMD, or CPU-only) and adjusts expectations accordingly. You get honest performance numbers for YOUR hardware — not a data centre benchmark.

No Vendor Lock-in

All embedding models come from open HuggingFace repositories. All LLMs run through open-source Ollama. You own your evaluation pipeline entirely.

08 // TECHNOLOGY STACK

Built on Proven Open-Source Tools

Every component of the AI Benchmarking Dashboard is open source, actively maintained, and free to use. The stack was chosen to maximise local performance, ease of installation, and accuracy of results.

LAYER	TECHNOLOGY	PURPOSE
Frontend	React 18 + Vite + Tailwind CSS	Dark-themed, fast, responsive UI with live streaming progress
Backend	FastAPI (Python)	High-performance REST + SSE streaming API
Embeddings	sentence-transformers (HuggingFace)	Local inference — no API keys needed for benchmarking
Scoring	ranx	Industry-standard IR metrics: MRR, NDCG, Recall
PDF Parsing	pdfplumber / pypdf	Robust text extraction from any PDF structure
QA Gen	OpenRouter (Claude / GPT)	Synthetic question generation — only external API call
LLM Runtime	Ollama	Run Llama, Mistral, DeepSeek and more locally
System Info	psutil + pynvml + rocm-smi	Live CPU, RAM, VRAM gauges during benchmarks
Charts	Recharts	NDCG bar charts, t-SNE scatter plots, TPS graphs

SUMMARY

The AI Benchmarking Dashboard turns a complex, expensive, and time-consuming process — evaluating which AI model is right for your documents — into a simple, local, and free workflow. Upload a PDF, pick your models, click Run. The tool does the rest.

Whether you are an engineer choosing between embedding models, a researcher validating retrieval quality, or a product team assessing LLM speed on real hardware, this dashboard gives you the data you need to make confident, evidence-based decisions — without sending your data anywhere.

github.com/naku2001/Metrics — MIT License — Free & Open Source